



Indian Institute of Information Technology, Vadodra International Campus Diu

Voice Assisatant (ZIRA)

GROUP-7

Under the guidance of
Dr. Deepika Gupta

November 22, 2024

Group Members

Sr. No.	Name	Roll No.
1	Abhi Trivedi	202311001
2	Saransh Naik	202311077
3	Kshitiz Gangwar	202311045
4	Pradeep Kumar Saran	202311066
5	Shailendra Singh Mandal	202311078
6	Pratham Popatiya	202311065

Outline

- 1 Introduction
- 2 About the code
 - Functionalities of code
 - Libraries used
 - OOPs concepts used
- 3 Benefits of OOPs in the Project
- 4 Use-case diagram
- 5 Class diagram
- 6 Future enhancements

Introduction

The Voice Assistant project, "Zira" is an AI-powered desktop assistant designed to simplify tasks using object-oriented programming. It combines speech recognition, text-to-speech synthesis, and automation for a seamless user experience.

The project structure follows a modular and extensible approach, using abstract command classes to define functionality, ensuring scalability for future enhancements. Additionally, the integration of a logging system allows tracking of executed commands for transparency and debugging.

Zira not only showcases practical applications of object-oriented principles but also demonstrates the power of combining programming with real-world utility.

Functionalities of code

- **Time and Date:** Provides current time and date.
- **Wikipedia Search:** Fetches topic summaries.
- **Web Automation:** Opens popular websites like Google and YouTube.
- **Music Playback:** Plays songs from a local directory.
- **Weather Updates:** Provides real-time weather information.
- **WhatsApp Messaging:** Sends instant messages.
- **Screenshots:** Captures and saves screenshots.
- **Entertainment:** Shares jokes and riddles.

Libraries used

- **pyttsx3**

- **Purpose:** Converts text to speech for voice responses.
- **Application:** Used for Zira's verbal communication with the user.

- **speech_recognition**

- **Purpose:** Converts speech to text for command recognition.
- **Application:** Enables Zira to understand and process user queries.

- **wikipedia**

- **Purpose:** Fetches summaries of topics from Wikipedia.
- **Application:** Provides concise and accurate information on user-specified topics.

- **webbrowser**

- **Purpose:** Opens websites in the default browser.
- **Application:** Automates access to sites like Google, YouTube, and Stack Overflow.

- **pywhatkit**

- **Purpose:** Facilitates WhatsApp messaging and other quick tasks.
- **Application:** Sends instant messages on WhatsApp.

Libraries used

- **pyautogui**
 - **Purpose:** Automates GUI interactions, such as taking screenshots.
 - **Application:** Captures and saves screenshots with a timestamp.
- **random**
 - **Purpose:** Provides randomization capabilities.
 - **Application:** Selects random songs or riddles when requested.
- **datetime**
 - **Purpose:** Handles date and time operations.
 - **Application:** Provides current date and time information.
- **requests**
 - **Purpose:** Sends HTTP requests for API calls.
 - **Application:** Fetches weather data and jokes from online APIs.
- **os**
 - **Purpose:** Interacts with the operating system.
 - **Application:** Opens files, directories, and handles file paths.
- **psutil**
 - **Purpose:** Monitors system processes.
 - **Application:** Checks if the music application is running.

- **Classes and Objects**

- Classes define blueprints for objects (e.g., 'VoiceAssistant', 'Command').
- Objects are created from classes to encapsulate data and methods.

- **Encapsulation**

- Bundling data and methods within classes.
- Example: 'VoiceAssistant' contains 'speak' and 'listen' methods.

- **Inheritance**

- Reuse common functionality through a base class.
- Example: 'TimeCommand' inherits from 'Command' and overrides 'execute'.

OOPs concepts used

- **Polymorphism**

- Same interface, different behavior.
- Example: 'Command' class defines 'execute', overridden in subclasses like 'WikipediaCommand'.

- **Abstraction**

- Hiding implementation details while exposing functionality.
- Example: User interacts with 'Command' classes through 'execute' without knowing internal details.

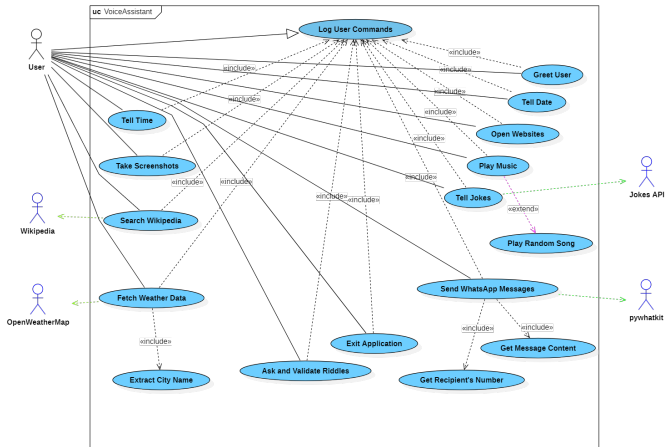
- **Composition**

- Objects composed of other objects for functionality.
- Example: 'TaskManager' uses 'VoiceAssistant' to execute commands.

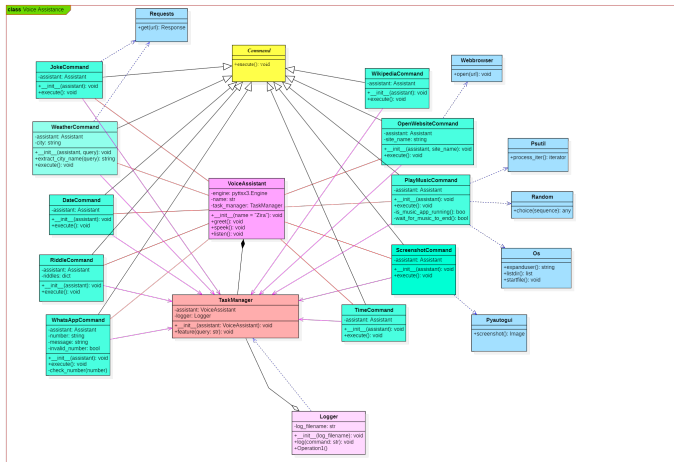
Benefits of OOPs in the Project

- **Modularity:** Code is organized into self-contained classes.
- **Scalability:** New features can be added by extending base classes.
- **Reusability:** Common functionality reused via inheritance.
- **Maintainability:** Encapsulation and abstraction simplify debugging.

Use-case diagram



Class diagram



Future enhancements

① Enhanced Natural Language Understanding (NLU):

- Integrate advanced NLP models like GPT for better query handling.
- Add multi-lingual support and context-aware interactions.

② Integration with IoT Devices:

- Enable control of smart home devices (lights, thermostats, appliances).
- Voice commands for managing IoT ecosystems.

③ Personalized User Experience:

- Implement user profiles for personalized greetings and routines.
- Introduce custom voice options and mood-based interactions.

④ Offline Functionality:

- Support basic tasks like playing music or managing files offline.
- Use local speech recognition models for privacy and efficiency.

⑤ Data Privacy and Security:

- Implement end-to-end encryption for sensitive data.
- Allow user-controlled data logs and transparent data management.

Thank You!

Questions?